

Welcome! We will begin shortly



AI-Assisted Coding

Learning Outcomes

- Apply 8-step AI workflow end-to-end for a given task
- Evaluate AI-generated solutions systematically by defining expected outcomes, identifying what to test, and checking relevant edge cases
- Debug and improve AI-generated code effectively by comparing actual results with expectations and tracing errors to their root causes rather than applying surface-level fixes.

AI-Assisted Coding with Codex in VS Code

No coding background needed.
One 8-step loop, and you will run it yourself.

In This Session

The Two Hours

- 01 The mindset: why your judgment matters
- 02 Setup, ground rules, and what we will build
- 03 The 8-step loop, run live once
- 04 Your turn: run the loop yourself
- 05 The clinical answer and your playbook

Learning Outcomes

After this session, you will be able to:

- ▶ Run the 8-step loop end to end, on any task
- ▶ Ask for one small piece at a time, and say why
- ▶ Read AI-written code back in plain language
- ▶ Decide what to check, including the edge cases
- ▶ Compare what you got against what you expected
- ▶ Trace an error to its cause instead of patching it

**You run the loop yourself in the second half.
Live AI is unpredictable. Reviewing the outputs is the core skill.**

Five Words We Will Use Today

Piece

One small part of the tool that does one job. Programmers call it a function. We will say piece.

Prompt

What you type to the AI. A request, in ordinary English. There is nothing to memorise.

.py and .md

clinic_tracker.py holds the work the computer runs.
project_notes.md holds the decisions you made.

Check

A small trial that proves a piece behaves. You will see: expected 240.00, got 240.00, PASS.

Drift: when the AI does more than you asked for. Say the word out loud when you spot it.

Part 1: The Mindset

Your judgment matters more than syntax

Why This Matters More in Healthcare

The quiet failure

- ▶ Code that runs but is wrong is like a test result that prints cleanly but was measured incorrectly
- ▶ Nothing crashes. It simply looks trustworthy.
- ▶ When work falls behind, checking is the first step people cut
- ▶ We are not cutting it here

Why you catch what a coder might not

- ▶ The people who spot AI errors knew the answer before they asked
- ▶ You already form that expectation every day, clinically
- ▶ So we agree the expected answer first
- ▶ Then we have something to check against

Plausible is not correct. Form the expectation before you read the output.

You Are the Director, Not the Typist



The model

A consultant directing a resident owns the final decision, even though the resident did the write-up.

You set the goal, you check the work, you own the call.



The collaborator

Treat AI as a confident but forgetful junior colleague.

Fast and willing. Occasionally wrong with total conviction. Forgets between sessions.



Green zone, amber zone

Lean in: summarising a file, drafting a first version, explaining it back to you.

Slow down: anything you cannot verify, or anything going into a real workflow.

Never: real patient information, at any point.

If the AI wrote it, you own it. Synthetic data only, never real patient information.

Part 2: Setup and What We Will Build

The workspace, the guardrails, and the number you write down first

Setup: Codex in VS Code

Three areas, three verbs

ASK

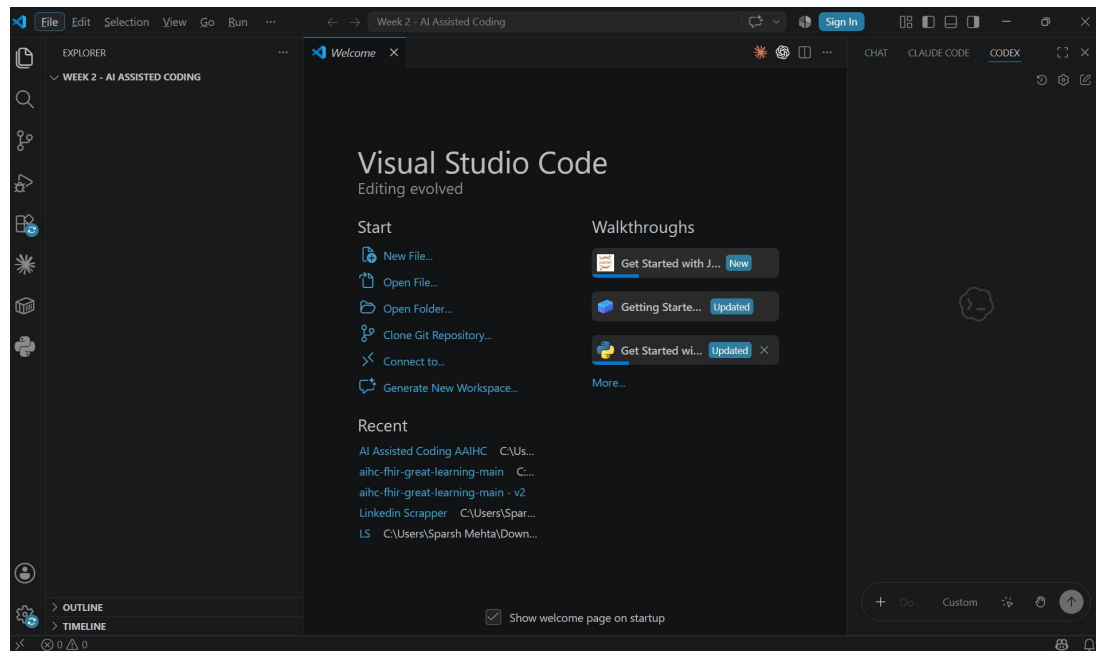
The chat panel. Where you talk to the AI.

WRITE

The editor. Where the work is held.

RUN

The terminal. Where you run it and see the result.



Generation speed varies. Pauses are normal.

Ground Rules and Our Safety Net

Rule zero: made-up data only

Never use real patient information, at any point.
Not in a prompt, not in a file, not in a screenshot.

The one-minute rule

If you cannot read it aloud in a minute, it is too big. Ask for a smaller piece.

Read before you run

Every time, even when you are confident it is fine. The habit is the point.

There is a safety net

Prepared versions of every step exist. A slow response becomes a paste, not dead time.

Synthetic data only. Never real patient information, at any point.

Step 1: What We Will Build, and What It Will Show

Version 1 WILL include

- ▶ Add an appointment: reference, date, category, cost
- ▶ Total spending by category
- ▶ Save to a file and reopen it later
- ▶ Print a summary a clinic manager can read

Version 1 will NOT include

- ▶ No interface
- ▶ No email or notifications
- ▶ No real data, no database
- ▶ Naming this stops the AI running ahead

Write this down now: Consultation is two visits at 120.00. You should see 240.00 at the end.

The 8-Step Loop

1

Define the goal

2

Ask for a plan

3

Convert to Tasks

4

Implement One

5

Add notes as you go

6

Generate Tests

7

Debug and Review

8

Refactor and Repeat

**Attributed to Dr Ian McCulloh. One piece per pass.
“Build me everything” gives you code you cannot check.**

Part 3: Run the Loop, Pass 1

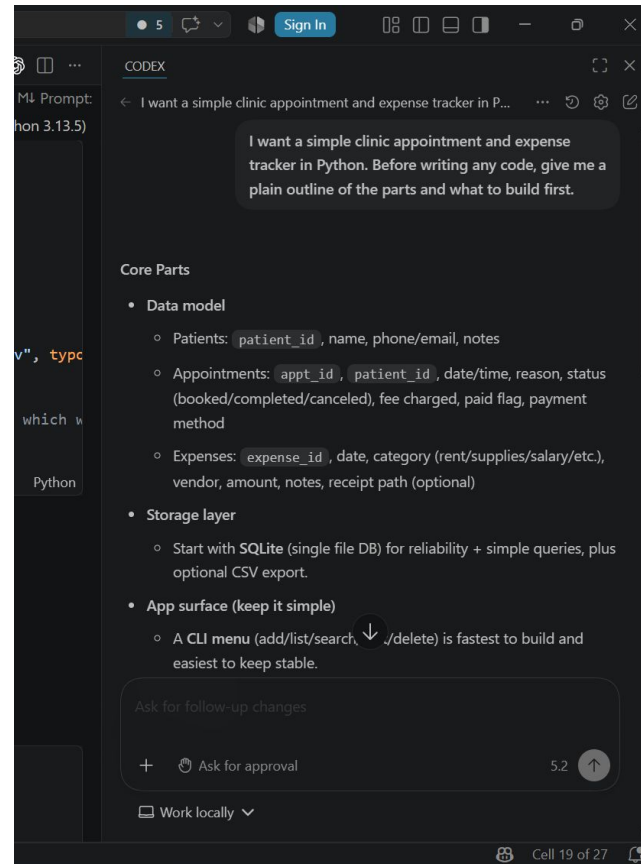
Step 2 Live: Ask for a Plan, No Code Yet

SUGGESTED AI PROMPT

I want a simple clinic appointment and expense tracker in Python. Before writing any code, give me a plain outline of the parts and what to build first.

This is where you steer

- ▶ Does this outline match what we agreed in Step 1?
- ▶ Saving is in. An interface is out.
- ▶ Add or remove parts now, before anything is built
- ▶ The words “before writing any code” are doing the work. Delete them and it starts typing.



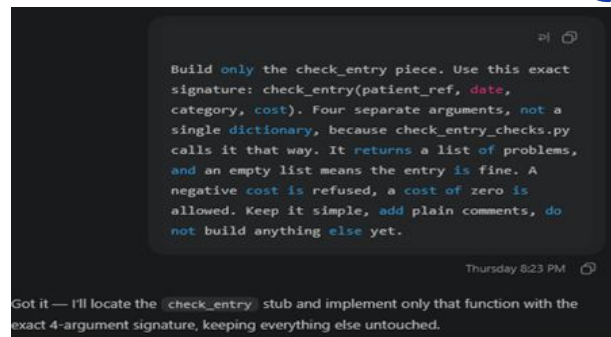
Step 3 Live: Break It Into Small Pieces

SUGGESTED AI PROMPT

Create an empty outline of the tool with each part named but not yet built, and a short note on what each part should do. Do not write the working code yet.

A named-but-empty outline is a to-do list

- ▶ Count the pieces against the plan from Step 2
- ▶ Five names, five notes, no work done
- ▶ If it builds anyway, that is drift. Stop there.
- ▶ Before I answer it: what would you say back to the AI? Say it out loud, then we try it.



Absolutely — here is `check_entry` explained line by line in plain language.

Line-by-Line (Plain Language)

- `clinic_tracker.py:1`: Starts a top note (a description block) for this file.
- `clinic_tracker.py:2`: Blank line for readability.
- `clinic_tracker.py:3`: Says this file currently only has the `check_entry` part.
- `clinic_tracker.py:4`: Ends that top note.
- `clinic_tracker.py:5`: Blank line.
- `clinic_tracker.py:6`: A human comment: this next part lists allowed categories.
- `clinic_tracker.py:7`: Starts a set/list of valid category names.
- `clinic_tracker.py:8`: Adds "Consultation" as allowed.
- `clinic_tracker.py:9`: Adds "Diagnostics" as allowed.
- `clinic_tracker.py:10`: Adds "Procedure" as allowed.
- `clinic_tracker.py:11`: Adds "Follow-up" as allowed.
- `clinic_tracker.py:12`: Adds "Pharmacy" as allowed.
- `clinic_tracker.py:13`: Closes the allowed-category set.
- `clinic_tracker.py:14`: Blank line.
- `clinic_tracker.py:15`: Blank line.
- `clinic_tracker.py:16`: Defines a function named `check_entry` with 4 inputs: `patient_ref`, `date`, `category`, `cost`.
- `clinic_tracker.py:17`: Starts a function description note.
- `clinic_tracker.py:18`: Blank line inside that note.
- `clinic_tracker.py:19`: Explains return rule: list of problems; empty list means valid.
- `clinic_tracker.py:20`: Ends function. ↓ caption note.
- `clinic_tracker.py:21`: Creates an empty `problems` list to collect any issues

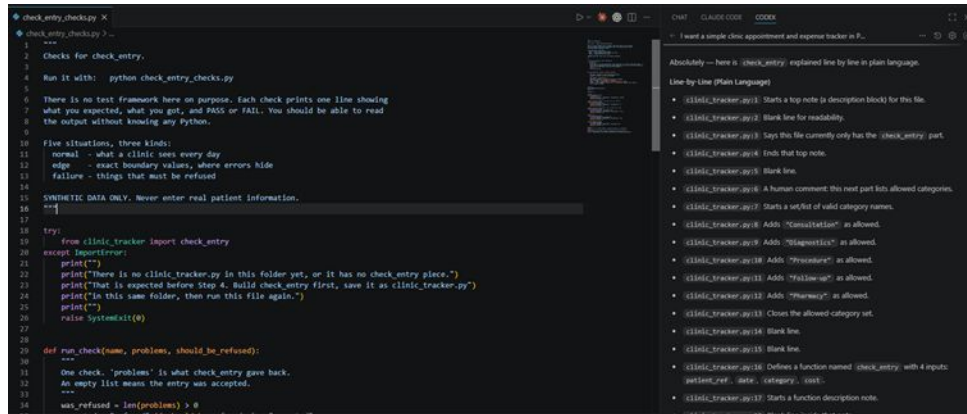
Step 4 Live: Build One Piece, Predict, Then Run

SUGGESTED AI PROMPT

Build only the check_entry piece. It takes a patient reference, a date, a category and a cost, and returns a list of problems. An empty list means the entry is fine. A negative cost is refused, a cost of zero is allowed. Keep it simple, add plain comments, do not build anything else yet.

Predict first, then run it

- ▶ Before we run it: what should happen with a cost of minus 50? Say it out loud.
- ▶ Now run it on one made-up entry and see whether you were right
- ▶ Did it assume anything we did not say? A date format? A maximum cost?
- ▶ Improve once, then stop



The screenshot shows a code editor with a Python script on the left and an AI-generated explanation on the right. The script is a simple Python program that checks if a patient entry is valid based on certain criteria. The AI explanation breaks down the script line by line, explaining the purpose of each line and the overall logic of the program.

```

1  # check_entry.py
2  """
3  Checks for check_entry.
4  Run it with: python check_entry.py
5  """
6  There is no test framework here on purpose. Each check prints one line showing
7  what you expected, what you got, and PASS or FAIL. You should be able to read
8  the output without knowing any Python.
9
10 Five situations, three kinds:
11 normal - what a clinic sees every day
12 edge - exact boundary values, where errors hide
13 failure - things that must be refused
14
15 SYNTHETIC DATA ONLY. Never enter real patient information.
16 """
17
18 try:
19     from clinic_tracker import check_entry
20 except ImportError:
21     print("")
22     print("There is no clinic_tracker.py in this folder yet, or it has no check_entry place.")
23     print("That is expected before Step 4. Build check_entry first, save it as clinic_tracker.py")
24     print("In this same folder, then run this file again.")
25     print("")
26     raise SystemExit(0)
27
28
29 def run_check(name, problems, should_be_refused):
30     """
31     One check. 'problems' is what check_entry gave back.
32     An empty list means the entry was accepted.
33     """
34     was_refused = len(problems) > 0
  
```

The AI explanation on the right side of the editor provides a line-by-line breakdown of the script. It starts with a top-level description of the script's purpose, then goes through each line, explaining its function. For example, it explains that line 1 is a comment, line 2 is a docstring, line 3 is a function call, and so on. It also explains the logic of the try-except block and the run_check function.

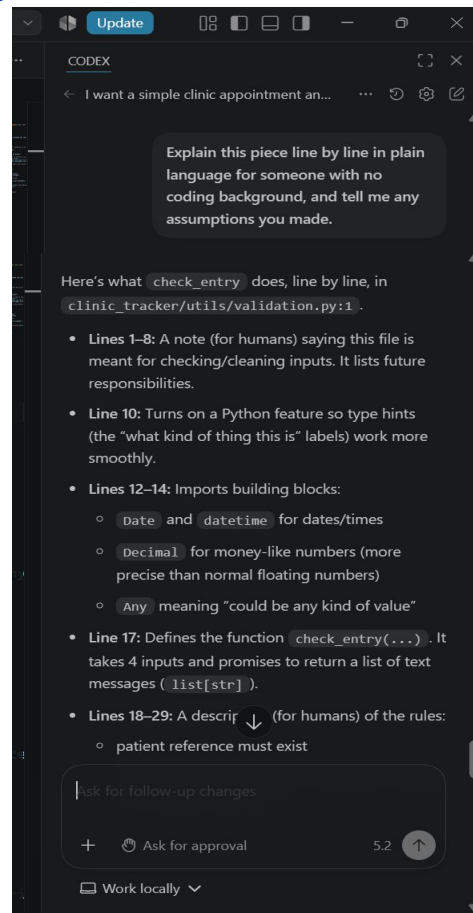
Step 4 Live: Read It Back in Plain Language

SUGGESTED AI PROMPT

Explain this piece line by line in plain language for someone with no coding background, and tell me any assumptions you made.

Restate it in one sentence

- ▶ Put the whole piece into one sentence in your own words
- ▶ For example: "It checks four things and refuses the entry if any are wrong."
- ▶ Then pick one claim the explanation makes and check it against the actual line
- ▶ The explanation itself can be wrong. If you cannot manage one sentence, slow down.



Step 5 Live: Write the Decisions Down

THE TWO FILE TYPES, AND WHY THE DIFFERENCE MATTERS

project_notes.md holds the decisions.
clinic_tracker.py holds the work.

Tricky cases we decide on purpose

- ▶ A cost of zero is ALLOWED. Zero means free, not invalid.
- ▶ A negative cost is REFUSED, not quietly accepted.
- ▶ A missing reference or date is REFUSED.
- ▶ An unknown category is REFUSED.
- ▶ A decision that only lives in the chat window is one the AI will forget.

```

project_notes.md X
AAIHC_Week2_MLS_v2 > learner_pack > project_notes.md > abc # Clinic Tracker: project notes
1  # Clinic Tracker: project notes
39 ## What comes out
43
44 ---
45
46 ## Tricky cases we decided on purpose
47
48 These are our decisions, not the AI's. If the AI does something
   different, the AI is wrong.
49
50 - A cost of zero is **allowed**. Zero means free, not invalid.
   In the sample month, 21 post-operative follow-ups inside the
   global period are recorded at no charge.
51 - A negative cost is **refused**, not quietly accepted.
52 - A missing patient reference or date is **refused**.
53 - An unknown category is **refused**.
54 - An empty list totals to nothing. It does not crash.
55 - A category with no activity shows as zero in the report. It
   does not disappear.
56
57 ---
58
59 ## The pieces
60
61 Built in this order, because each one depends on the one above
   it.
62
63 1. 'check_entry' refuse bad input before it gets in
64 2. 'add_appointment' add one appointment to the list
65 3. 'total_by_category' add up spending per category
66 4. 'save' write the list to a file
67 5. 'load' read the list back later
68 6. 'build_report' turn the totals into something a clinic
   manager would read

```

Step 6: Check It. Normal, Edge, Failure

Normal cases

A typical appointment entry
The totals a clinic sees every day

Edge cases

An empty list
A single entry
An exact boundary value, like a
cost of zero

Failure cases

An unknown category
A negative cost that must be
refused

**The same AI that built the feature can write a check that repeats its own mistake.
You decide what to test.**

Step 6 Live: Run the Checks and Test the Edges

SUGGESTED AI PROMPT

Write checks for check_entry covering: a good entry, a cost of exactly zero, a cost of minus 0.01 just below zero, a clearly negative cost, and an unknown category. For each one, print the name, what you expected, what you got, and PASS or FAIL. No test framework, just printed lines. Then run: `python check_entry_checks.py`

Five checks, five printed lines

- Five situations in, five lines out. Count them on screen: nothing is hidden.
- Each line reads: expected, got, PASS or FAIL. No commands to decode.
- Now one by hand, on paper. Consultation is 120 plus 120. You expected 240.
- Zero is allowed, minus 0.01 is refused. That boundary is where errors hide.

```
python check_entry_checks.py

CHECKS FOR check_entry

a good entry                expected accepted got accepted PASS
a cost of exactly zero      expected accepted got accepted PASS
a cost of minus 0.01, just below zero expected refused got refused PASS
a clearly negative cost of minus 50 expected refused got refused PASS
an unknown category         expected refused got refused PASS

5 of 5 checks passed.

Now do one by hand, on paper, before you trust any of this:
```

Spot the Flaw: Six Minutes, No Hints

A Version A

- ▶ Goes through the appointments looking for the category
- ▶ Returns the cost as soon as it finds a match
- ▶ Nothing crashes. It prints one clean number.
- ▶ Consultation: 120.00

B Version B

- ▶ Goes through every appointment, adding up each match
- ▶ Returns the running total at the end
- ▶ Nothing crashes. It prints one clean number.
- ▶ Consultation: 240.00

You wrote 240.00 down in Step 1. Which one is wrong, why, and would a passing check have told you?

Step 7: Fix and Improve

Read the message

It says where and what. Read it once, properly, before doing anything else.

Ask for the cause

Not just a patch. Pasting “fix this” and hoping wastes more time than reading the error once.

Check it against your notes

Compare the result to `project_notes.md`, not to your memory of the conversation.

Re-run everything

Not only the check that failed. A fix, or an improvement, can disturb something else.

Never accept a fix, or an improvement, that you cannot explain.

Step 7 Live: Break It, Then Fix It Properly

SUGGESTED AI PROMPT

Here is the `check_entry` function, the failing check, and the exact error message. Explain the root cause of the failure, then give a corrected version with comments.

Predict which checks will fail

- ▶ The slip: change `if cost_value < 0:` to `if cost_value < -1000:`
- ▶ Before we run: how many of the five will fail, and which ones? Commit out loud.
- ▶ Two fail. Nothing crashes. The tool runs and quietly accepts what it should refuse.
- ▶ Did it explain the cause, or hand you a patch? Then re-run all five, not just the two.

CHECKS FOR `check_entry`

a good entry	expected	accepted	got	refused	FAIL
a cost of exactly zero	expected	accepted	got	refused	FAIL
a cost of minus 0.01, just below zero	expected	refused	got	refused	PASS
a clearly negative cost of minus 50	expected	refused	got	refused	PASS
an unknown category	expected	refused	got	refused	PASS

3 of 5 checks passed.

Step 7 Live: A Cleaner Version, Same Behaviour

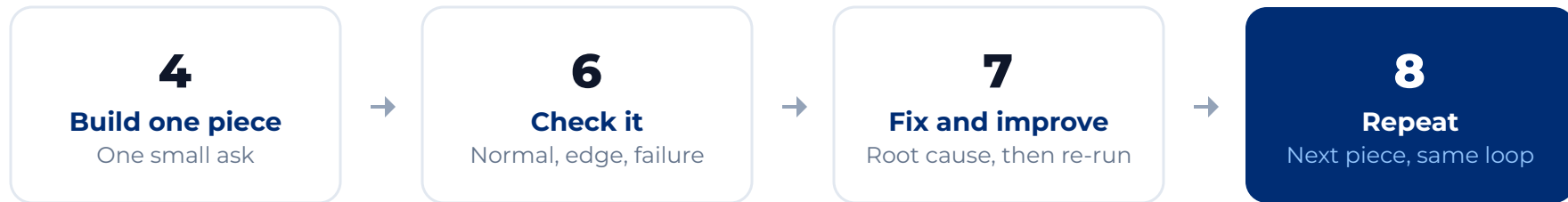
SUGGESTED AI PROMPT

Review this function for clarity, efficiency and good practice, then give an improved version that behaves exactly the same.

Prove the behaviour is unchanged

- ▶ Twenty-five lines become six
- ▶ Tidier is not the claim. Identical behaviour, proved by the checks, is the claim.
- ▶ Re-run every check. Green.
- ▶ If a check fails, the AI changed behaviour while claiming it did not. The checks caught it.

Step 8: Repeat the Loop for the Next Piece



STEP 5 Add notes as you go: update `project_notes.md` the moment a decision is made, at any point in the pass.

Carry forward into the next pass

- `project_notes.md`: the decisions do not change.
- The checks you already wrote. Keep running them.
- Your expected answers, worked out by hand.

Start fresh each pass

- One new piece only. **Never two.**
- A new small ask, readable in a minute.
- "Do not build anything else yet."

One pass on `check_entry` is done. In the next part you run the loop yourself.

Part 4: Your Turn, Then the Clinical Answer

Your Turn: Run the Loop on add_appointment

What to do, 25 minutes

- ▶ Step 4: build only add_appointment, using the BUILD prompt. Nothing else.
- ▶ Step 5: read it back in one sentence, in your own words.
- ▶ Step 6: choose three situations to check, and write your expected answer first.
- ▶ Step 7: if a check fails, ask for the cause, fix it, then re-run all three.

What we will ask you afterwards

- ▶ Which step were you on when it went wrong?
- ▶ Did it build anything you did not ask for? What did you say back?
- ▶ Did any check pass that should have failed?
- ▶ Was there anything you could not explain in plain language?

Work in pairs. One types, one reads aloud and challenges. Swap over at Step 6.

Five Pieces, Five Passes

STEP 8 OF 8

PASS 1

check_entry

Refuse bad input
before it gets in

built live together

PASS 2

add_appointment

Add one
appointment to the
list
you built this one

PASS 3

total_by_category

Add up spending
per category
same loop, run again

PASS 4

save / load

Write to a file, read
it back later
same loop, run again

PASS 5

build_report

Turn totals into a
clinic summary
same loop, run again

You have now run the loop twice, once watching and once yourself. Passes 3 to 5 were the same loop again, with the same decisions file and the same checks. Nothing new was learned between pass one and pass five.

The pieces are now correct. Next we turn them into an answer someone can act on.

Close the Loop: Run It for the Clinician

RUN IN THE TERMINAL

```
python clinic_report.py
```

What you should see

- ▶ 7 appointments recorded, 1,755.75 total
- ▶ Consultation 240.00 | Diagnostics 550.50 | Procedure 890.00
- ▶ Follow-up 60.00 | Pharmacy 15.25
- ▶ Largest cost driver: Procedure, at 50.7% of spend

SPEND SUMMARY

Category	Visits	Spend	Share
Consultation	2	240.00	13.67%
Diagnostics	2	550.50	31.35%
Procedure	1	890.00	50.69%
Follow-up	1	60.00	3.42%
Pharmacy	1	15.25	0.87%
TOTAL	7	1755.75	100.00%

Largest cost driver: Procedure (890.00)

Why This Step Is the Point

Five passing checks is not the win

- ▶ A green terminal is a developer's outcome
- ▶ It proves the tool is checkable, not that it is correct
- ▶ No clinic manager has ever asked to see a test run

This is the win

- ▶ A summary someone can act on
- ▶ Procedure is half the spend. That is a decision, not a number.
- ▶ You verified 240.00 by hand, so you can defend the 1,755.75

You expected 240 and you got 240. That is why you can trust the total.

Part 5: Scaling Up and Your Playbook

What changes when projects get bigger and the AI gets more autonomous

Bigger Projects and the Agentic Angle

Agentic AI means several AI helpers working in sequence, handing work to each other.



What stays true at any size

- ▶ The AI forgets between sessions. The notes file is its memory and yours.
- ▶ Named placeholders are a to-do list, not a failure.
- ▶ Check for repeated logic every few steps, before it hardens.

Your Playbook: 8 Steps, 8 Prompts

01 DEFINE · 02 PLAN

"Here is the goal. It will do these things, and not these things."

"Before any code, give me a plain outline of the parts and what to build first."

03 BREAK DOWN · 04 BUILD

"Split this into small pieces, each doable in one request."

"Build only this one piece. Simple, plain comments, nothing else yet."

05 NOTE · 06 CHECK

"Add these decisions to project_notes.md, in plain language."

"Check this piece for these situations. Print expected, got, PASS or FAIL."

07 FIX AND IMPROVE · 08 REPEAT

"Here is the code and the exact error. Explain the cause, then correct it."

"Now the next piece only. Same notes file, same checks. Nothing else yet."

"Do not build anything else yet" is your best defence against the AI running ahead.

Practise This Yourself

1. Open the pack

README.md explains what each file is for, and why .md and .py have different jobs.

2. Read the decisions

project_notes.md. Change one rule, then see which checks fail.

3. Run one more pass

Pick total_by_category. Run steps 4, 6 and 7 on it, exactly as you did in the session.

4. Check your answer

python clinic_report.py should print 1,755.75 across 7 appointments.

Synthetic data only. Never real patient information, at any point.

Key Takeaways

You are the director

Set the goal, check the work,
own the call.
If the AI wrote it, you own it.

One piece, one pass, always checked

Eight steps, run in order.
Nothing longer than a minute to
read.
Test the edges, not just the easy
cases.

Plausible is not correct

Explain it in plain language.
Verify a result by hand.
Keep real patient data out of
every prompt.

Synthetic data only. Never real patient information, at any point.



Power Ahead!

